ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл, холбооны технологийн сургууль



БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Үйлдийн систем IV-2-4

[F.CSM302]

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр: **Kernel Data Structures**

Бие даалтын ажлыг гүйцэтгэсэн оюутан:

Г. Үлэмж /B232270805/

Б. Лхамрагчаа /B241940102/

Т. Буянтөгс /B231910034/

Э. Төгсбаяр /221910021/

Ө. Чинзориг /200900028/

Б. Идэрбат /B242270075/

Хичээл заасан багш : **Г.ГАНБАТ /F.CS10/**

Улаанбаатар 2026 он

Гарчиг

Хураангуй 2

1. Удиртгал 2

1.1 Зорилго 2

1.2 Ач холбогдол 3

2. Сурах бичгийн шаардлага 3

3. Онолын үндэс 4

3.1 Дугуй хоёр холбоост жагсаалт 4

3.2 Linux-ийн арга 4

3.3 Collatz дараалал 4

4. Системийн зохиомж 4

4.1 Бүтэц 4

4.2 Шалгалт ба алдаа 5

5. Хэрэгжилт 5

5.1 Хэл ба орчин 5

5.2 Part I — `color_list.c` 5

5.3 Part II — `collatz.c` 5

5.4 Shim 5

5.5 Engine ба сервер 5

5.6 UI 5

6. Туршилт ба UI дүрслэл 6

6.1 Юуг өөрчилж турших вэ 6

6.2 Юу ажиглах вэ 6

7. Туршилтын арга 7

7.1 Орчин 7

7.2 Хүлээгдсэн утга хэрхэн гарсан 7

7.3 Ажиллуулах 7

8. Үр дүн 7

8.1 `make test` 7

8.2 Номын жишээ (Part I) 8

8.3 Part II — Collatz лавлагаа 8

8.4 Алдаа ба захын оролт 8

8.5 UI-ийн бодит screenshot 8

8.6 Автомат UI шалгалт ба бие даасан QA 10

8.7 Модулийг жинхэнэ кернелийн header-тэй хөрвүүлсэн нь 10

8.8 Жинхэнэ кернелд ачаалсан үр дүн 11
9. Хэлэлцүүлэг 12
9.1 Үр дүнгийн тайлбар 12
9.2 Цаашдын ажил 12
10. Дүгнэлт 13
Ашигласан материал 13
Хавсралт А. Ажлын төлөвлөгөө 13
Хавсралт Б. Файлууд ба хэмжээ 14

Хураангуй

Асуудал. Сурах бичгийн 3-р бүлгийн 4-р төслийн хүрээнд kernel-ийн дугуй хоёр холбоост жагсаалт ашиглан өнгөний жагсаалт болон Collatz дараалллыг хэрэгжүүлэх шаардлагатай байсан. Мөн даалгаврын нэг хэсэг болгон үйлдэл хийх бүрд жагсаалтын бүтэц хэрхэн өөрчлөгдөж байгааг графикаар харуулах хэрэгтэй болсон.

Шийдэл. Kernel module-ийн үндсэн эх кодыг өөрчлөхгүйгээр хоёр өөр орчинд шалгасан. Эхлээд kernel-ийн шаардлагатай функцүүдийг дуурайлган хийсэн жижиг `shim` давхарга ашиглаж, кодыг энгийн user-space программ хэлбэрээр ажиллуулсан. Дараа нь ижил эх кодыг ажиллаж байгаа kernel-ийн жинхэнэ header ашиглан `.ko` module болгон хөрвүүлсэн. Үүний дараа багийн гишүүн module-уудыг өөрийн машин дээр kernel-д ачаалж, ажиллагааг нь шалгасан.

Жагсаалтын үндсэн ажиллагааг хариуцах C engine, Python сервер болон веб UI хийсэн. UI дээр харагдах жагсаалтын графикийг тусад нь зохиомлоор үүсгээгүй бөгөөд тухайн мөчид C engine дотор байгаа бодит жагсаалтын төлвөөс шууд зурдаг байдлаар хийсэн. Ингэснээр нэмэх, устгах зэрэг үйлдлийн дараа жагсаалт бодитоор хэрхэн өөрчлөгдөж байгааг харах боломжтой болсон.

Туршилт. `make test` ажиллуулахад 4 C тест, 9 scene тест болон 12 HTTP тест ажиллана. C түвшний тестүүдийг ажиллуулахдаа санах ойн алдаа болон тодорхойгүй үйлдлийг илрүүлэхийн тулд AddressSanitizer болон UndefinedBehaviorSanitizer ашигласан.

Мөн `make module` командаар хоёр kernel module-ийг жинхэнэ kernel header ашиглан `.ko` файл болгон амжилттай хөрвүүлсэн. Энэ хэсгийн үр дүнг 8.7-д оруулсан. Үүний дараа багийн гишүүн өөрийн машин дээр `insmod` ашиглан module-уудыг kernel-д ачаалж, `dmesg`-ийн гаралтыг шалгасан бөгөөд үр дүнг 8.8-д харуулсан.

Гол үр дүн. Одоогоор хэрэгжүүлсэн бүх тест амжилттай дууссан (`All available tests passed`). Номонд өгөгдсөн 4 өнгө зөв гарсан. Collatz дараалын хувьд 27-оос эхлэхэд 112 зангилаа, 837799-өөс эхлэхэд 525 зангилаа үүссэн нь хүлээгдэж байсан үр дүнтэй таарсан.

Санах ойн ажиллагааг шалгахад allocation хийсэн бүх санах ойг зөв буцааж чөлөөлсөн бөгөөд leak илрээгүй. Үүнээс гадна allocation алдааг 1,254 өөр байрлалд зориудаар үүсгэж шалгасан. Жагсаалтын ажиллагааг нэмэлтээр шалгахын тулд 5,000 санамсаргүй үйлдэл үүсгэн, үр дүнг энгийн массив дээр хийсэн reference загвартай харьцуулсан.

Өнгөний өгөгдлийг шалгах үед номонд байгаа `violet` өнгийн green болон blue утгууд хоорондоо солигдсон байсныг олсон. Тиймээс стандарт RGB утга болох `(138, 43, 226)`-г ашигласан.

Эцэст нь module-ийг жинхэнэ kernel дээр ажиллуулахад өнгөний 4 мөр зөв гарсан. Collatz 27-ийн хувьд нийт 112 утга үүсэж, хамгийн их утга нь 9232 байсан. Энэ үр дүн нь өмнө нь `shim` орчинд ажиллуулж авсан үр дүнтэй ижил байсан.

1. Удиртгал

1.1 Зорилго

Linux kernel дотор олон төрлийн өгөгдлийг холбоост жагсаалтаар удирддаг. Тэдгээрийн нэг нь дугуй хоёр холбоост жагсаалт бөгөөд элемент бүр өмнөх болон дараагийн элемент рүүгээ заагчтай байдаг. Мөн жагсаалтын хамгийн сүүлийн элемент нь эхний элементтэйгээ холбогддог.

Энэ ажлын гол зорилго нь сурах бичгийн даалгаврын дагуу уг жагсаалтын бүтцийг хэрэгжүүлж, элемент нэмэх, устгах зэрэг үйлдэл хийх бүрд жагсаалтын холбоосууд хэрхэн өөрчлөгдөж байгааг графикаар ойлгомжтой харуулах юм.

1.2 Ач холбогдол

Холбоост жагсаалт нь Linux kernel-ийн олон хэсэгт ашиглагддаг. Жишээлбэл процессуудыг удирдах, ажлын дараалал үүсгэх, санах ой болон бусад kernel-ийн дотоод өгөгдлийг зохион байгуулахад хэрэглэнэ.

Ийм бүтцийг практик дээр хэрэгжүүлж үзэх нь kernel өгөгдлөө хэрхэн зохион байгуулдаг, pointer-ууд ямар үүрэгтэйг илүү сайн ойлгоход тусална. Ялангуяа kernel орчинд pointer-ийн алдаа нь системийн ажиллагаанд шууд нөлөөлөх эрсдэлтэй байдаг. Тиймээс энэ төсөлд жагсаалтын холбоосууд зөв байгаа эсэхийг тусгай шалгалт болон санах ойн алдаа илрүүлэх хэрэгслүүдээр давхар шалгасан.

2. Сурах бичгийн шаардлага

Эх сурвалж: *Operating System Concepts*, 10-р хэвлэл, 3-р бүлгийн «Project 4 — Kernel Data Structures» ба Exercise 3.21 (Collatz). Шаардлага бүрийн хэрэгжилтийг доорх хүснэгтэд харуулав.

Хүснэгт 1. Programming Project 4-ийн шаардлага ба биелэлт

№	Шаардлага	Хэрэгжилт	Төлөв
1	`struct color` (red, blue, green) дотор `list_head`; `static LIST_HEAD(color_list)`	`src/kernel/color_list.c`	Хэрэгжсэн
2	Модулийн эхэнд 4 өнгийг `kmalloc`-оор аваад жагсаалтын төгсгөлд нэмэх	ижил файл; `test_color_module` дараалал, тоог шалгана	Shim ✓ · кернел ✓
3	Жагсаалтыг гүйлгэж `printk`-ээр хэвлэх (dmesg)	хэвлэсэн мөр байтаар таарсан (8.2)	Shim ✓ · кернел ✓
4	Дуусахдаа `list_del` ба `kfree`-ээр чөлөөлөх	тестэд амьд зангилаа 0	Shim ✓ · кернел: rmmod алдаагүй
5	Part II: `collatz` модуль, `start` параметр (анхдагч 25)	`src/kernel/collatz.c`; 25 → 24 зангилаа	Shim ✓ · кернел: start=27
6	Collatz дараалллыг жагсаалтад хадгалж, хэвлэж, дуусахдаа чөлөөлөх (n тэгш бол $n/2$, сондгой бол $3n+1$)	7 лавлагаа Python-оор бие даан бодсон дараалалтай таарсан (8.3)	Shim ✓ · кернел: start=27
7	Багшийн UI: Kernel structure graph (нэмэх, устгах, хайх → граф)	`src/ui/static/*` ба engine	Хэрэгжсэн

`Shim ✓` тэмдэглэгээ нь тухайн бүтэц болон ажиллагааг shim орчинд автомат тестээр шалгасныг илэрхийлнэ. Харин `Кернел ✓` гэдэг нь модулийг жинхэнэ Linux kernel-д ачаалж, `dmesg`-ийн гаралтыг shim орчинд гарсан үр дүнтэй харьцуулан таарсныг баталгаажуулсан гэсэн үг. Энэ kernel шалгалтыг багийн гишүүн гүйцэтгэсэн бөгөөд дэлгэрэнгүйг 8.8 хэсэгт оруулсан.

Kernel дээр санах ойг `kfree` ашиглан чөлөөлсөн бөгөөд `rmmod` команд алдаагүй дууссан. Гэхдээ санах ойн leak-ийг тоогоор `0` гэж баталгаажуулсан шалгалт зөвхөн shim орчинд хийгдсэн. Өөрөөр хэлбэл kernel дээр нөөц зөв чөлөөлөгдөж байгааг ажиллагааны түвшинд шалгасан боловч `kmemleak` зэрэг хэрэгслээр тусад нь хэмжээгүй.

Мөн анхдагч `start=25` утгыг kernel дээр тусад нь туршаагүй. Kernel module-ийг ачаалахдаа `start=27` утга өгч шалгасан. Module-ийг жинхэнэ kernel-д ачаалахаас өмнө `.ko` файл болгон амжилттай хөрвүүлсэн бөгөөд энэ алхам 8.7 хэсэгт тэмдэглэгдсэн.

Үүнээс гадна БД1 Guide-д тавигдсан нийтлэг шаардлагуудыг мөн хэрэгжүүлсэн. Тухайлбал оролтын утгыг өөрчилж дахин турших, буруу оролтын үед ойлгомжтой алдааны мэдээлэл өгөх, өөрсдийн нэмэлт тестүүдийг хийх, мөн Manual-ийн дагуу төслийг дахин ажиллуулах боломжийг шалгасан. Эдгээрийн дэлгэрэнгүйг 6 болон 8-р бүлэгт оруулсан.

3. Онолын үндэс

3.1 Дугуй хоёр холбоост жагсаалт

Жагсаалтын элемент бүр өөрийн өгөгдлөөс гадна `next` болон `prev` гэсэн хоёр заагчтай байна. `next` нь дараагийн элементийг, харин `prev` нь өмнөх элементийг заана. Жагсаалтын хамгийн сүүлийн элементийн `next` нь буцаад жагсаалтын толгой руу заадаг учраас бүх элемент хоорондоо тойрог хэлбэрээр холбогдоно.

Энэ бүтэц нь элемент нэмэх болон устгах үед давуу талтай. Учир нь тухайн элементийн ойролцоох хоёр элементийн холбоосыг өөрчлөхөд хангалттай бөгөөд бүх жагсаалтыг дахин зохион байгуулах шаардлагагүй. Харин тодорхой утгатай элементийг хайх үед жагсаалтын элементүүдийг эхнээс нь дараалаар шалгаж явна.

3.2 Linux-ийн арга

Linux kernel-д холбоост жагсаалтыг хэрэгжүүлэхдээ `struct list_head` гэсэн жижиг бүтцийг тухайн өгөгдлийн бүтцийн дотор оруулж ашигладаг. Ингэснээр нэг ижил жагсаалтын функцүүдийг өөр өөр төрлийн өгөгдлийн бүтэц дээр ашиглах боломжтой болдог.

Жагсаалтын `list_head` хэсгээс тухайн үндсэн бүтцийг олохдоо `list_entry` макро ашигладаг. Өөрөөр хэлбэл жагсаалтын код нь өгөгдлийн төрлөөс шууд хамаарахгүй ажиллаж чаддаг.

Үйлдлүүдийн хугацааны төвөгшлийг Хүснэгт 2-т харуулсан. `O(1)` хугацаатай үйлдэл гэдэг нь жагсаалтад хэдэн элемент байгаагаас үл хамааран ойролцоогоор ижил хэмжээний ажил хийнэ гэсэн үг. Харин `O(n)` гэдэг нь жагсаалтын элементүүдийн тоо нэмэгдэх тусам хийх ажлын хэмжээ мөн дагаж нэмэгдэнэ гэсэн үг.

Хүснэгт 2. Жагсаалтын үйлдлүүд

Үйлдэл	Kernel API	Хугацаа
Толгойд эсвэл төгсгөлд нэмэх	`list_add`, `list_add_tail`	O(1)
Заагдсан элементийг устгах	`list_del` (дараа нь `kfree`)	O(1)
Утгаар хайх, гүйлгэх	`list_for_each_entry`	O(n)
Гүйлгэж байхдаа устгах	`list_for_each_entry_safe`	O(n)

3.3 Collatz дараалал

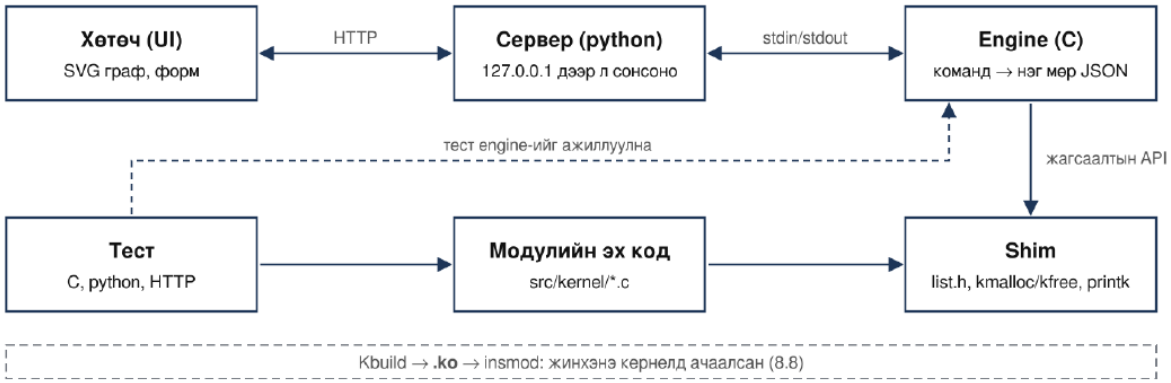
Collatz дараалал нь эерэг бүхэл `n` утгаас эхэлнэ. Хэрэв `n` тэгш бол 2-т хуваана, харин сондгой бол `3n + 1` үйлдлийг хийнэ. Энэ үйлдлийг `n = 1` болтол давтана.

Дараалал хэдэн алхам үргэлжлэх нь урьдчилан тодорхой биш учраас утгуудыг холбоост жагсаалтад хадгалах нь тохиромжтой. Мөн зарим эхлэх утгын хувьд дарааллын утга маш хурдан өсдөг. Жишээлбэл 837799-өөс эхлэхэд нийт 525 зангилаа үүсэж, хамгийн их утга нь 2,974,984,576 хүрнэ. Энэ нь 32 битийн тэмдэгтэй бүхэл тооны дээд хязгаар болох 2,147,483,647-оос их байна.

Үүнээс ч том утгаас эхлэхэд ялгаа улам их гарна. Жишээлбэл 2,147,483,647-оос эхлэхэд дарааллын хамгийн их утга 1,235,346,792,567,892 хүрдэг. Иймээс Collatz-ийн утгуудыг хадгалахдаа 32 битийн төрөл хангалтгүй гэж үзэж, 64 битийн бүхэл тоон төрөл ашигласан.

4. Системийн зохиомж

4.1 Бүтэц



Зураг 1. Системийн бүтэц. Engine болон жинхэнэ модулиуд нэг shim-ийг ашигладаг тул график дээрх заагч нь модулийн кодын заагчтай ижил механизмаар үүснэ. Тасархай хүрээ: модулийг Kbuild-ээр хөрвүүлж, багийн гишүүн жинхэнэ кернелд ачаалсан (8.8).

Нэг үйлдэл хийх үед систем дараах байдлаар ажиллана. Эхлээд хэрэглэгч UI дээр хийх үйлдлээ сонгоно. Сервер тухайн үйлдлийг engine-д нэг мөр команд хэлбэрээр дамжуулна. Үүний дараа engine жагсаалтын API-г бодитоор ажиллуулж, үр дүнг нэг мөр JSON хэлбэрээр буцаана.

UI талд графикийг тусад нь тооцоолж эсвэл зохиомлоор үүсгэдэггүй. Харин engine-ээс ирсэн JSON хариуг ашиглан тухайн үеийн жагсаалтын бодит төлвийг зурна. Ингэснээр UI дээр харагдаж байгаа бүтэц нь engine доторх жагсаалттай шууд таарч байна.

Командын хэлбэр болон оролтын жишээг Manual-ын 8.1.4 хэсгээс, харин алдааны код болон тэдгээрийн тайлбарыг 8.1.8 хэсгээс харж болно.

4.2 Шалгалт ба алдаа

Engine нь үйлдэл бүрийн дараа жагсаалтын холбоос зөв байгаа эсэхийг `kds_verify` шалгагчаар шалгана. Үүнд `next` болон `prev` заагчууд хоорондоо зөв таарч байгаа эсэх, мөн жагсаалтыг урагш болон хойш чиглэлээр тоолоход элементийн тоо ижил гарч байгаа эсэхийг шалгадаг.

Хэрэв буруу команд орж ирвэл жагсаалтын төлөв болон санах ойн тоолуур өөрчлөгдөх ёсгүй. Харин жагсаалтын бүтэц эвдэрсэн тохиолдолд engine гацахгүйгээр алдааг илрүүлж, UI дээр тухайн асуудлыг улаан `?` тэмдгээр харуулна.

Программ дуусах үед чөлөөлөгдөөгүй зангилаа үлдсэн эсэхийг мөн шалгана. Хэрэв санах ойд зангилаа үлдсэн байвал `LEAK` гэсэн мэдээлэл гаргаж, программ `3` гэсэн буцах кодтой дуусна.

5. Хэрэгжилт

5.1 Хэл ба орчин

C (`-std=gnu11 -Wall -Wextra -Werror`), GNU make, python3 (зөвхөн стандарт сан), цэвэр JavaScript ба SVG. Бусад хэрэглэсэн гадны сангуудын хувьд, framework, CDN, фонт ашиглаагүй. Файлуудын хэмжээг Хавсралт Б-д харуулав.

5.2 Part I — `color_list.c`

Номонд өгсөн алхмын дагуу `struct color` бүтцийн дотор `list_head`-ийг оруулж хэрэгжүүлсэн. Эхлээд 4 өнгийн мэдээлэлд `kmalloc` ашиглан санах ой авч, дараа нь `list_add_tail` функцээр жагсаалтын төгсгөлд дараалаар нь нэмсэн. Үүссэн жагсаалтын утгуудыг `printk` ашиглан kernel log дээр хэвлэж шалгасан.

Module ажиллаж дуусах үед жагсаалтын элемент бүрийг `list_del` ашиглан салгаж, өмнө нь авсан санах ойг `kfree` ашиглан чөлөөлсөн.

Номын жишээг шалгах үед `violet` өнгийн green болон blue утгууд `struct`-ийн талбарын дараалгатай холбоотойгоор солигдсон байсныг анзаарсан. Тиймээс хэрэгжүүлэлтэд зөв RGB утга болох `(138, 43, 226)`-г ашигласан.

5.3 Part II — `collatz.c`

Хэрэв `start` утга 1-ээс бага байвал оролтыг буруу гэж үзэж алдаа буцаана. Kernel орчинд 64 битийн утгыг 2-т хуваахын оронд `value >>= 1` битийн шилжилт ашигласан. Энэ нь тухайн утгыг 2-т хуваасантай ижил үр дүн өгнө.

Collatz-ийн дараагийн утгыг тооцоолох үед 64 битийн бүхэл тооны хязгаараас давах эрсдэл гарвал тооцооллыг үргэлжлүүлэхгүйгээр `-EINVAL` буцаана. Харин шинэ зангилаанд санах ой авах үед `kmalloc` амжилтгүй болбол тухайн хүртэл үүсгэсэн бүх зангилааг чөлөөлж, `-ENOMEM` алдааг буцаадаг байдлаар хийсэн. Ингэснээр алдаа гарсан үед өмнө нь авсан санах ой үлдэхээс хамгаалсан.

5.4 Shim

`Shim` нь kernel-ийн жинхэнэ header файлуудын оронд тестийн үед ашиглагддаг жижиг давхарга юм. Ингэснээр kernel module-ийн эх кодыг өөрчлөхгүйгээр user-space орчинд ажиллуулж, логикийг нь шалгах боломжтой болсон.

Shim дотор `list.h`, `kmalloc/kfree`, `printk` болон санах ойн алдааг зориудаар үүсгэхэд хэрэгтэй функцүүдийг хэрэгжүүлсэн. `list.h`-ийг Linux-ийн эх кодоос шууд хуулж аваагүй бөгөөд номонд тайлбарласан зарчмын дагуу өөрсдөө бичсэн. `kmalloc` болон `kfree` нь авсан, чөлөөлсөн санах ойг тоолдог. Мөн санах ойг чөлөөлсний дараа тусгай утгаар дарж, нэг санах ойг дахин `kfree` хийх зэрэг алдааг шууд илрүүлэх боломжтой болгосон.

Kernel module-ийн эх файлууд дотроо хэвийн байдлаар `#include <linux/...>` ашигладаг. Тест ажиллах үед эдгээр include-ийг shim-ийн header-ууд хангадаг тул модулийн эх кодыг тусгайлан өөрчлөх шаардлагагүй. Харин жинхэнэ kernel module хөрвүүлэх үед `Kbuild` нь яг ижил эх файлуудыг kernel-ийн бодит header-уудтай хамт хөрвүүлдэг. Ингэснээр shim дээр шалгасан болон жинхэнэ kernel-д ашигласан код нь нэг эх код хэвээр үлдсэн.

5.5 Engine ба сервер

Engine нь нэг команд хүлээж аваад, тухайн командыг нэг мөр JSON хариу болгон буцаадаг байдлаар ажиллана. Сервер нь зөвхөн `127.0.0.1` хаяг дээр хүсэлт сонсож, нэг engine процессыг удирдана.

Хэрэв engine процесс ямар нэг шалтгаанаар дууссан байвал дараагийн хүсэлт ирэх үед сервер engine-ийг дахин эхлүүлж, ажиллагааг үргэлжлүүлнэ. Ингэснээр серверийг бүхэлд нь дахин асаах шаардлагагүй.

Серверийг терминалаас `Ctrl+C` дарж зогсоох үед engine-ийн оролтыг хаана. Үүний дараа engine өөрийн цэвэрлэгээний ажиллагааг хийж, үлдсэн санах ой байгаа эсэхийг шалгаад хэвийн унтарна.

5.6 UI

UI нь өнгө, Collatz, стек, дараалал гэсэн нийт 4 табтай. Таб бүр дээр тухайн бүтэцтэй ажиллах үйлдлийн форм, жагсаалтын график, санах ойн ашиглалтыг харуулах тоолуур, хийсэн үйлдлүүдийн алхамчилсан түүх болон алдааны мэдээлэл харагдана.

График дээр жагсаалтын холбоосуудыг ялгаж харахад хялбар байхаар дүрсэлсэн. `next` заагчийг бүтэн шугамтай сумаар, харин `prev` заагчийг тасархай шугамтай сумаар харуулдаг. Ингэснээр элементүүд хоорондоо аль чиглэлд холбогдож байгааг графикаас шууд харах боломжтой.

UI-ийн ашиглалт болон үйлдлүүдийн дэлгэрэнгүй тайлбарыг Manual-ын 8.1 хэсгээс үзэж болно.

6. Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж турших вэ

Систем дээр дараах үндсэн үйлдлүүдийг хийж болно.

> Өнгөний хувьд RGB утгыг `0–255` мужид өгч, жагсаалтын эхэнд, төгсгөлд эсвэл тодорхой индексийн дараа нэмэх боломжтой. Мөн индексээр эсвэл өнгөөр устгах, хайлт хийх үйлдлүүдийг дэмжинэ.
>
> Collatz дараалалд `start` утгыг `1–2147483647` мужид өгч болно. Жишээлбэл `25`, `27`, `837799` зэрэг утгуудаар дараалал үүсгэж, дарааллаас утгаар хайх болон устгах боломжтой.
>
> Стек дээр `push`, `pop`, `peek`, харин дараалал дээр `enqueue`, `dequeue`, `peek` үйлдлүүдийг хийж болно.
>
> Нэмэлтээр `--max-nodes` ашиглан үүсэх зангилааны дээд хязгаарыг тохируулж болно. `inject_fail after=N` командаар санах ойн allocation алдааг зориудаар үүсгэж шалгах боломжтой. Харин `reset` командаар тухайн үеийн бүх өгөгдлийг цэвэрлэнэ.

6.2 Юу ажиглах вэ

- **График:** Шинээр зангилаа нэмэхэд тухайн зангилаа ялгарч харагдана. Мөн `next` болон `prev` холбоосын сумнууд шинэчлэгдэж, жагсаалтын төгсгөлөөс толгой руу буцсан нум зурагдсанаар бүтэц нь тойрог хэлбэртэй болохыг харуулна.
- **Санах ойн тоолуур:** Шинэ зангилаа үүсэх бүрд `kmalloc` тоолуур нэмэгдэж, зангилаа устахад `kfree` тоолуур нэмэгдэнэ. Тухайн үед амьд байгаа зангилааны тоог `kmalloc - kfree` гэж харуулна.
- **Алдаа:** Алдаа гарсан үед UI дээр алдааны код болон тайлбар гарч ирнэ. Энэ үед жагсаалтын өмнөх төлөв болон санах ойн тоолуур өөрчлөгдөхгүй хэвээр үлдэнэ. Үүнийг Зураг 4-т харуулсан.

7. Туршилтын арга

7.1 Орчин

Зүйл	Бодитоор ашигласан хувилбар
OS	Arch Linux, Linux 7.2.11-arch1-1 x86_64
Compiler	gcc (GCC) 16.2.1 20260810; AddressSanitizer + UndefinedBehaviorSanitizer
Build	GNU Make 4.4.1
Runtime	Python 3.14.8 (стандарт сан)
Browser	Brave 156 (headless; screenshot, UI шалгалт)
Кернелд ачаалсан	Linux 7.2.11-arch1-1, багийн гишүүний машин (8.8)
Кернелийн header	linux-headers 7.2.11.arch1-1; зөвхөн `.ko` хөрвүүлэхэд (`make test` хэрэглэдэггүй)

7.2 Хүлээгдсэн утга хэрхэн гарсан

Тестийн хүлээгдсэн утгуудыг engine-ийн гаралтаас шууд хуулж ашиглаагүй. Collatz дараалыг Python ашиглан тусад нь бодож, дараа нь номонд өгөгдсөн мэдэгдэж буй үр дүнтэй тулгаж шалгасан. Жишээлбэл 27-оос эхлэхэд 111 алхам хийгдэж, хамгийн их утга нь 9232 болох ёстойг бие даан баталгаажуулсан.

Жагсаалтын ажиллагааг шалгахдаа reference болгон энгийн массив загвар ашигласан. Нийт 5,000 санамсаргүй үйлдэл хийж, үйлдэл бүрийн дараа холбоост жагсаалтын төлөв массивын төлөвтэй яг ижил байгаа эсэхийг харьцуулсан. Ингэснээр зөвхөн эцсийн үр дүнг биш, үйлдэл бүрийн дараах жагсаалтын өөрчлөлтийг шалгасан.

7.3 Ажиллуулах

```
make clean && make && make test     # бүх тест; алдаатай бол 0-ээс өөр код
make run                            # UI: http://127.0.0.1:8080
# жинхэнэ кернелийн header-тэй .ko хөрвүүлнэ (ачаалалт: 8.8):
make module KDIR=/usr/lib/modules/$(uname -r)/build
```

8. Үр дүн

8.1 `make test`

Тестэн дээр (`build/` байхгүй) `make && make test` нийт ≈ 2.5 секунд (хөрвүүлэлт орсон; 5 удаа хэмжихэд 2.45–2.54 с), exit code 0. Эцсийн мөр: `All available tests passed.`

Хүснэгт 3. make test-ийн бодит үр дүн

Тест	Юуг шалгасан	Үр дүн
`test_list`	Өөрийн `list.h`: 5,000 санамсаргүй үйлдэл (seed `0x0c011a72`) массив загвартай адил, алхам бүрт холбоос зөв, leak 0	**PASS**
`test_color_module`	Жинхэнэ `color_list.c`: `printk` гаралт байтаар таарсан; 4 өнгө; `kmalloc` = `kfree`; 4 байрлалд санах ойн алдаа	**PASS**
`test_collatz_module`	Жинхэнэ `collatz.c`: 7 лавлагаа; 1,250 байрлалд санах ойн алдаа; 2 буруу `start`; leak 0	**PASS**
`test_core`	Engine-ийн бүх команд, алдаа, хязгаар, 512 байтын хил; шалгагч эвдэрсэн 10 жагсаалтыг барьсан	**PASS**
`run_cases.py`	9 сценарийн бүлэг: 229 хариуг хүлээгдсэн JSON-той тулгасан	**PASS**

Тест	Юуг шалгасан	Үр дүн
`test_server.py`	12 HTTP тест (`Ran 12 tests in 0.428s — OK`): статик файл, path traversal, loopback, engine үхэх, 24 зэрэгцээ команд, Ctrl+C-ийн цэвэр shutdown (SIGINT үл тоомсорлогдож өвлөгдсөн үед ч)	**PASS**

8.2 Номын жишээ (Part I)

`color` жишээг ажиллуулахад нийт 4 зангилаа үүсэж, `printk` ашиглан дөрвөн өнгийн мэдээлэл kernel log дээр хэвлэгдэнэ. `violet` өнгийн зөв RGB утга нь `(138, 43, 226)` байна.

Эхний төлөвт `kmalloc = 4`, `kfree = 0` бөгөөд нийт 4 амьд зангилаа байна. Нэг зангилаа 32 байт хэмжээтэй учраас нийт ашиглаж байгаа санах ой 128 байт болно.

Дараа нь `color delete index=1` үйлдэл хийхэд тухайн зангилаа жагсаалтаас устаж, `kfree` тоолуур 1 болно. Ингэснээр амьд зангилааны тоо 3 болж, ашиглаж байгаа санах ой 96 байт болж буурна.

```
color[0]: red=138, green=43, blue=226
color[1]: red=255, green=0, blue=0
color[2]: red=0, green=255, blue=0
color[3]: red=0, green=0, blue=255
```

8.3 Part II — Collatz лавлагаа

Хүснэгт 4. Жинхэнэ collatz.c (shim дээр): бүх лавлагаа таарсан

start	Зангилаа	Алхам	Оргил	Үр дүн
1	1	0	1	**PASS**
15	18	17	160	**PASS**
25 (анхдагч)	24	23	88	**PASS**
27	112	111	9,232	**PASS**
97	119	118	9,232	**PASS**
837,799	525	524	2,974,984,576	**PASS**
2,147,483,647	451	450	1,235,346,792,567,892	**PASS**

Санах ойн allocation-ийн алдааг зориудаар үүсгэж шалгасан. Үүний тулд `k`-р allocation дээр алдаа гаргахаар тохируулж, `k = 0`-оос эхлээд тухайн дараалалд үүсэх зангилааны тоо хүртэл бүх боломжит байрлалыг туршсан.

Алдаа гарсан тохиолдол бүрт модуль `-ENOMEM` буцааж, түүнээс өмнө амжилттай авсан бүх санах ойг зөв чөлөөлсөн. Ингэснээр тест бүрийн дараа leak `0` болсон. Collatz-ийн туршилтуудад нийт 1,250 allocation байрлал, өнгөний модульд 4 байрлал шалгасан бөгөөд нийлээд 1,254 өөр санах ойн алдааны нөхцөлийг туршсан.

Engine дээр `collatz run start=27` командыг ажиллуулахад нийт 112 зангилаа үүснэ. Нэг зангилаа 24 байт хэмжээтэй тул нийт ашиглагдах санах ой `24 × 112 = 2,688` байт болсон.

8.4 Алдаа ба захын оролт

Буруу оролт өгсөн үед engine нь `E_RANGE`, `E_BAD_VALUE`, `E_NOT_FOUND`, `E_EMPTY`, `E_FULL`, `E_NOMEM` зэрэг алдааны код болон тайлбарыг хамтад нь буцаадаг. Эдгээр тохиолдол бүрт жагсаалтын өмнөх төлөв болон `kmalloc` тоолуур өөрчлөгдөөгүй эсэхийг давхар шалгасан.

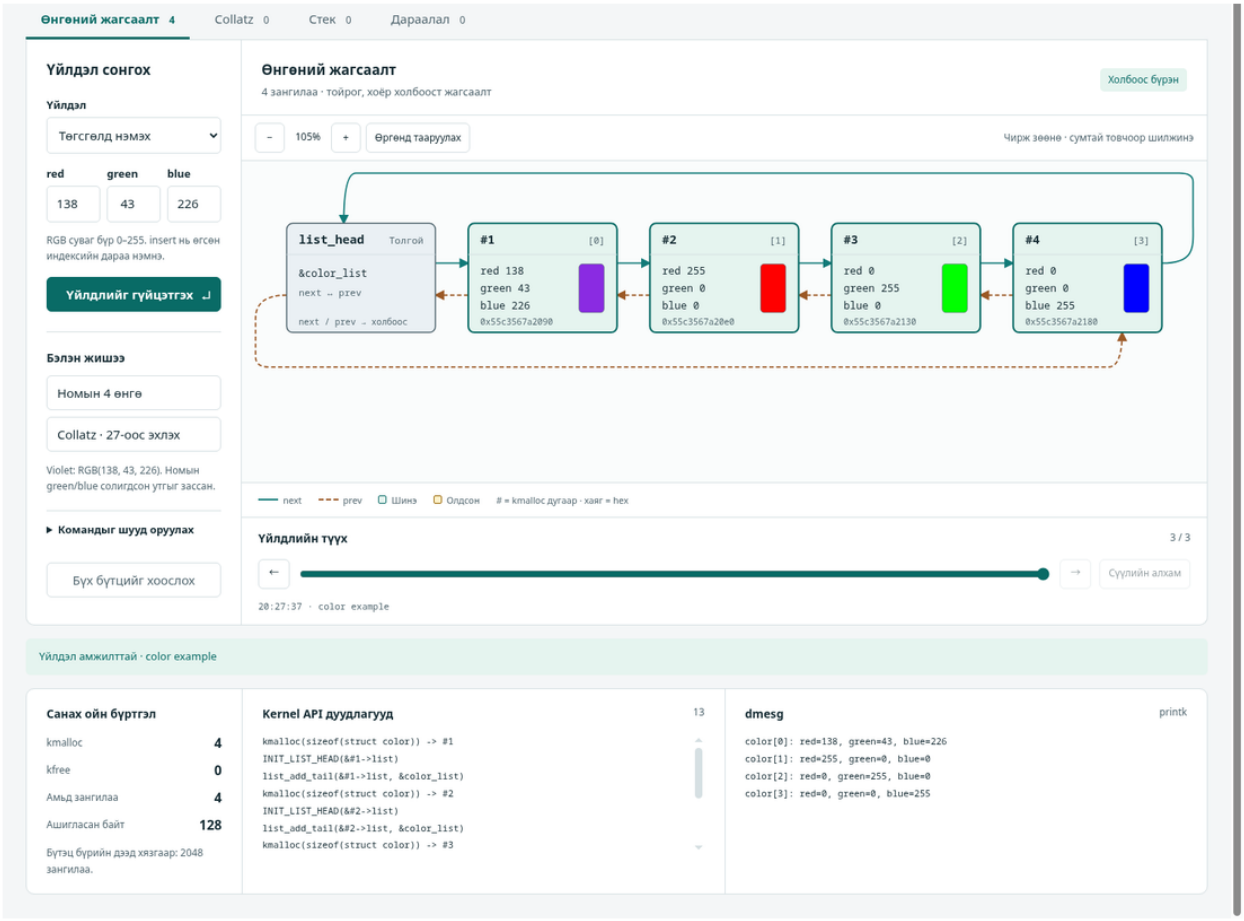
Мөн 600 тэмдэгттэй хэт урт мөр өгөхөд engine `E_SYNTAX` алдаа буцаасан боловч дараагийн командыг хэвийн боловсруулж чадсан. Үүнээс гадна 128 санамсаргүй binary мөрөөр туршихад engine унаагүй.

Алдааны кодын бүрэн жагсаалтыг Manual-ын 8.1.8 хэсгээс, харин захын болон нэмэлт тестүүдийг 8.2.6 хэсгээс харж болно.

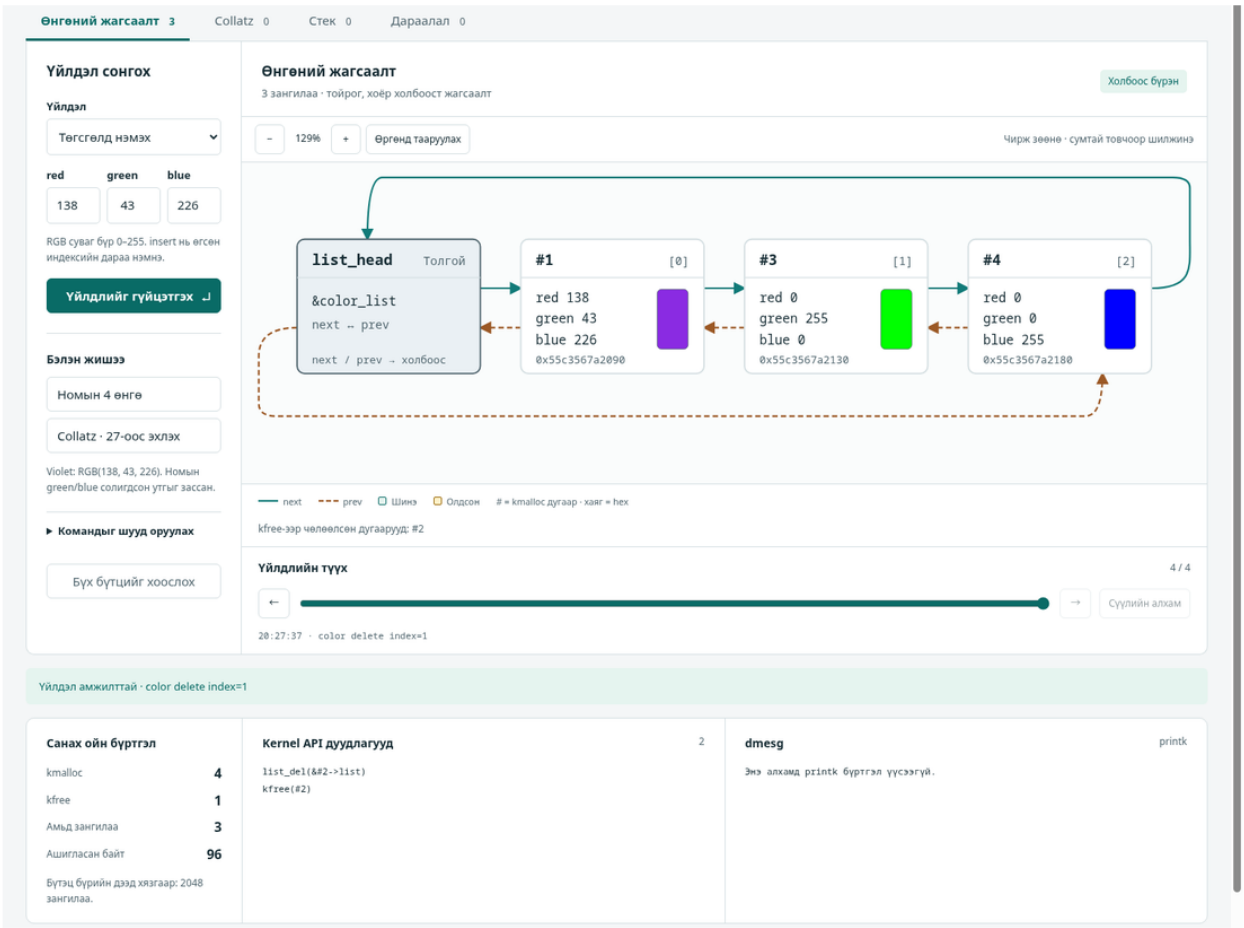
8.5 UI-ийн бодит screenshot

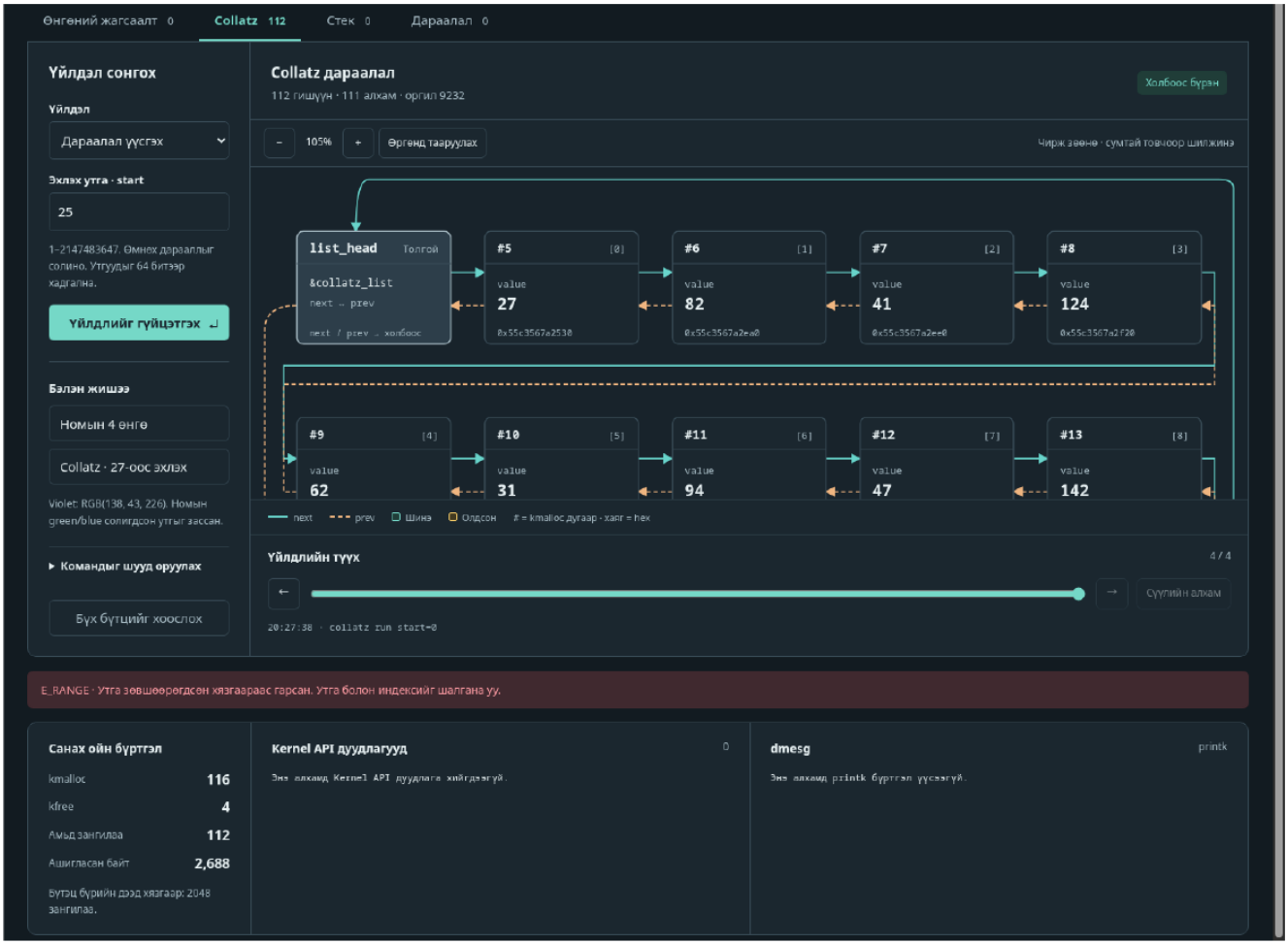
Доорх зургууд нь ажиллаж буй серверээс Brave (headless)-аар авсан бодит screenshot бөгөөд

Зураг 2. color example: толгойноос #1 → #2 → #3 → #4 → толгой руу next (тасралтгүй), prev (тасархай). Тоолуур: kmalloc 4, kfree 0, амьд 4, 128 байт.



Зураг 3. color delete index=1: #2 устсаны дараа #1 → #3 → #4 болж холбоос шинэчлэгдсэн; чөлөөлсөн дугаар (#2) графикийн доор жагсааж дүрсэлсэн байна. Тоолуур: kmalloc 4, kfree 1, бүтэн 3, 96 байт.





Зураг 4. Алдааны төлөв (dark горим): `collatz run start=0` → «E_RANGE · Утга зөвшөөрөгдсөн хязгаараас гарсан…». Өмнөх `collatz run start=27`-ийн график(112 зангилаа, зураг мөр мөрөөр байрлана) болон тоолуур (kmalloc 116 = 4 өнгө + 112, kfree 4, амьд 112, 2,688 байт) өөрчлөгдөөгүй. Командыг форм биш, `?run=`-аар илгээсэн: форм `start` талбарт `min=1` тул browser `0`-г өөрөө хаагдана.

8.6 Автомат UI шалгалт ба бие даасан QA

Багийн гишүүн Brave browser ашиглан UI-ийн үндсэн ажиллагааг туршиж шалгасан. Өнгө, Collatz, стек, дараалал гэсэн 4 таб бүгд хэвийн ажиллаж байсан.

Collatz 27-оос эхлүүлэхэд UI дээр 112 зангилаа болон 226 холбоосын сум зөв зурагдсан. Мөн хайлт хийх, элемент устгах зэрэг үндсэн үйлдлүүдийг туршиж, бодит жагсаалтын төлөвтэй зөв таарч байгааг шалгасан. Түүхийн горимд орсон үед жагсаалтыг өөрчлөх үйлдлүүд түгжигдэж байгаа эсэх, мөн алдаа гарсан үед алдааны төлөв UI дээр зөв харагдаж байгаа эсэхийг мөн шалгасан.

Responsive байдлыг шалгахдаа дэлгэцийн өргөнийг 320 px болон 390 px болгож туршихад хэвтээ scroll үүсээгүй. Мөн шалгалтын явцад browser-ийн console дээр JavaScript алдаа гараагүй.

Эдгээр UI тестийн скриптүүдийг `tests/ui/` хавтсанд хадгалсан. Тестүүд нь тухайн орчинд Brave эсвэл Chromium суулгасан байх шаардлагатай тул `make test` командын үндсэн тестийн багцад оруулаагүй болно.

Бие даасан QA шалгалт

Өөр нэг багийн гишүүн хөгжүүлсэн системийг тусдаа кодын чанарыг Manual-ийн зааврын дагуу дахин шалгалт хийсэн. Үүнд `make`, `make test`, `make run` командууд болон Manual-д өгөгдсөн `curl` жишээнүүдийг ажиллуулсан. Мөн дээр дурдсан хоёр UI тестийн скриптийг тусад нь дахин ажиллуулж, Хөгжүүлсэн багийн гишүүний гаргасан үр дүнтэй ижил үр дүн гарсныг баталгаажуулсан.

8.7 Модулийг жинхэнэ кернелийн header-тэй хөрвүүлсэн нь

`make module KDIR=/usr/lib/modules/$(uname -r)/build` командыг ажиллуулахад Kbuild хоёр модулийг амжилттай хөрвүүлсэн. Хөрвүүлэлтийн явцад `CC [M]`, `MODPOST`, `LD [M]`, `BTF [M]` гэсэн үндсэн алхмууд дараалаар хийгдэж, эцэст нь хоёр `.ko` файл үүссэн. Энэ үед алдаа болон анхааруулга гараагүй.

Модулийг хөрвүүлэхийн тулд эх кодод тусгай өөрчлөлт хийгээгүй. `make test`-ийн module тестүүд мөн яг ижил `color_list.c` болон `collatz.c` файлуудыг ашигладаг. Өөрөөр хэлбэл shim орчинд шалгасан код болон жинхэнэ kernel header ашиглан хөрвүүлсэн код нь нэг эх код байсан.

Хүснэгт 5. make module-ийн үр дүн (src/kernel/; gcc 16.2.1; linux-headers 7.2.11.arch1-1)

Модуль	Хэмжээ	license	vermagic	Параметр
`color_list.ko`	213,424 B	GPL	7.2.11-arch1-1 SMP preempt mod_unload	байхгүй
`collatz.ko`	359,088 B	GPL	7.2.11-arch1-1 SMP preempt mod_unload	`start` (int)

Энэ нь модулийн эх код ажиллаж буй кернелийн жинхэнэ API болон компайлертай нийцэж байгааг харуулна. Ачаалсан үр дүнг 8.8-д харуулав.

8.8 Жинхэнэ кернелд ачаалсан үр дүн

Доорх үр дүнг багийн гишүүн өөрийн машин дээр (Arch Linux, Linux 7.2.11-arch1-1 x86_64), өөрийн терминалаас, repository-ийн язгуур хавтаснаас гараар гүйцэтгэсэн. AI агентууд үүнийг давтаагүй, QA модулийг ачаалаагүй. Гаралтыг багийн гишүүний терминалаас хуулсан; screenshot хавсаргаагүй.

```
sudo insmod src/kernel/color_list.ko
sudo dmesg | tail -20
sudo rmmod color_list

sudo insmod src/kernel/collatz.ko start=27
sudo dmesg | tail -130
sudo rmmod collatz
```

`insmod` ба `rmmod` алдаа өгөөгүй; `rmmod`-ийн дараа `lsmod`-д хоёр модуль аль нь ч үлдээгүй. Хэвлэсэн лог (dmesg-ийн сүүл) дээр эдгээр модулийн анхааруулга, oops байгаагүй. `color_list` 4 мөр хэвлэсэн (зүүн); `collatz start=27` 112 мөр (индекс 0…111) хэвлэсэн, доор эхний ба сүүлийн мөрүүд, оргилын орчим (баруун):

```
color[0]: red=138, green=43, blue=226
color[1]: red=255, green=0, blue=0
color[2]: red=0, green=255, blue=0
color[3]: red=0, green=0, blue=255
```

```
collatz[0]: value=27
collatz[1]: value=82
collatz[2]: value=41
collatz[3]: value=124
...
collatz[76]: value=3077
collatz[77]: value=9232
collatz[78]: value=4616
...
collatz[110]: value=2
collatz[111]: value=1
```

Хүснэгт 6. Shim ба жинхэнэ кернелийн харьцуулалт

Зүйл	Shim (make test)	Жинхэнэ кернел (багийн гишүүн)
color_list	4 элемент; эхнийх нь violet (138, 43, 226); 4 мөр байтаар таарсан	4 мөр; эхнийх нь violet (138, 43, 226); мөр бүр shim-ийн хүлээгдсэн мөртэй адил
collatz start=27	112 утга, 111 алхам, оргил 9232	112 утга (индекс 0…111), 111 алхам, оргил 9232 (индекс 77)
Python лавлагаатай харьцуулалт	112 утга бүгд (`test_collatz_module`)	Энд хуулсан утгууд (индекс 0…8, 76…78, 109…111) таарсан; 112 мөрийг мөр мөрөөр бүрэн харьцуулаагүй
`insmod` / `rmmod`	Холбогдохгүй	Алдаагүй; `lsmod`-д үлдээгүй
Хөрвүүлэлт	gcc, `-Wall -Wextra -Werror`	Kbuild, gcc 16.2.1, linux-headers 7.2.11.arch1-1 (Хүснэгт 5)

Энэ хоёр үр дүн нь жинхэнэ Linux-ийн `list.h`, `kmalloc`, `printk` ашигласан модуль shim зохиомол давхарга дээр ажилласантай ижил гаралт ажилглагдсан: өнгөний 4 мөр, Collatz-ийн мөрийн тоо, индекс, толгой нь таарсан.

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

- **Зөв байдал:** бие даасан Python лавлагаа, массив загвар, холбоосын шалгагч гурвуулаа engine ба модулиудтай таарсан тул жагсаалтын API болон Collatz логик зөв гэдэгт итгэлтэй байна.

- **Санах ой алдагдаагүй:** амжилттай болон санах ойн алдаатай аль ч замд `kmalloc` = `kfree` (shim дээр).
- **Жинхэнэ кернел:** color_list ба collatz (start=27) ачаалахад өнгөний 4 мөр, Collatz-ийн мөрийн тоо, оргил shim-ийнхтэй таарсан (8.8).

9.2 Цаашдын ажил

Кернел дээр захын `start` (1, 0, сөрөг), анхдагч 25-г шалгах, kmemleak ашиглах, `rmmod`-ийн дараах `dmesg`-ийг хадгалах, ачаалалтыг screenshot-оор энэ тайланд нэмэх; UI-ийн автомат шалгалтыг `make test`-д холбох.

10. Дүгнэлт

Номын Part I болон Part II-д тавигдсан шаардлагуудыг kernel module-ийн жинхэнэ эх код дээр хэрэгжүүлсэн. Нэг ижил эх кодыг гурван өөр түвшинд шалгасан. Эхлээд user-space shim орчинд кодыг өөрчлөлтгүйгээр ажиллуулж функциональ байдлыг нь шалгасан. Дараа нь жинхэнэ kernel header ашиглан `.ko` module болгон амжилттай хөрвүүлсэн. Эцэст нь багийн гишүүн тухайн module-ийг жинхэнэ kernel дээр ачаалж үр дүнг баталгаажуулсан.

Мөн үйлдэл бүрийн дараах linked list-ийн бүтцийг engine доторх бодит pointer-уудаас уншиж харуулдаг интерактив UI хийсэн. Ингэснээр жагсаалтын төлөв хэрхэн өөрчлөгдөж байгааг шууд харах боломжтой болгосон.

Шалгалтын үр дүнд номонд заасан дөрвөн өнгө, тэр дундаа violet өнгө зөв гарч байгааг баталгаажуулсан. Collatz-ийн 7 reference case бүгд зөв ажилласан. Санах ойн allocation-ийг зориудаар 1,254 өөр байрлалд алдаатай болгож туршихад leak илрээгүй. Мөн зориудаар эвдсэн 10 linked list-ийг шалгагч зөв илрүүлсэн. Буруу input өгөх үед ойлгомжтой error code буцааж, өмнөх төлөв өөрчлөгдөхгүй байгааг мөн шалгасан.

`make test` командыг төслийн цэвэр хуулбар дээр ажиллуулахад бүх тест амжилттай дууссан. Хоёр kernel module хоёулаа жинхэнэ kernel header ашиглан `.ko` файл болж хөрвөсөн. Багийн гишүүн module-уудыг kernel-д ачаалж шалгахад өнгөний дөрвөн мөр болон Collatz 27-ийн 112 утга, хамгийн их утга болох 9232 нь user-space shim дээр гарсан үр дүнтэй таарсан.

Одоогоор kernel орчин дээр буруу болон хязгаарын `start` утгууд, `ENOMEM` нөхцөл, `kmemleak` ашигласан санах ойн алдагдлын шалгалтыг бүрэн хийгээгүй. Мөн өөр компьютер болон өөр kernel version дээр туршиж үзээгүй байгаа.

Дараагийн шатанд эдгээр дутуу тестүүдийг нөхөж хийх, төслийг GitHub-д байршуулж, Manual-ийн дагуу өөр компьютер дээр шинээр татаж авч дахин ажиллуулах шаардлагатай.

Ашигласан материал

1. Silberschatz, A., Galvin, P. B., Gagne, G. *Operating System Concepts*, 10th ed., Wiley, 2018 — §1.9 «Kernel Data Structures»; 3-р бүлэг, «Project 4 — Kernel Data Structures», Exercise 3.21 (Collatz).

2. F.CSM302 Үйлдлийн систем — *Бие даалт 1* заавар (OS_BD1_Guide.pdf) ба «Бие даалтын ажил 1-ын хуваарилалт» (OS_Topic_and_Peer_Review_Allocation.pdf).
3. Linux-ийн `<linux/list.h>` API (нэр, семантик; номд заасан) — код хуулаагүй, `src/shim/linux/list.h` өөрийн хэрэгжилт.
4. Хэрэгсэл: GCC 16.2.1, GNU Make 4.4.1, Python 3.14.8, AddressSanitizer/UndefinedBehaviorSanitizer, Brave 156 (headless). Төслийн `tests/README.md`.

Хавсралт А. Ажлын төлөвлөгөө

Хуваарилалтын код	IV-2-4
Төсөл	Kernel Data Structures (OS Concepts 10e, Ch. 3, Project 4)
Repository	[GitHub холбоос] (`kernel-data-structures`)
Хэл, орчин	C (gnu11), GNU make, python3 стандарт сан, JavaScript + SVG; Arch Linux; гуравдагч сан байхгүй
Үндсэн дүрслэл	Kernel structure graph

Гишүүд ба хариуцах ажил — санал болгосон хуваарилалт; баг эцэслэн батална. Commit/file баганад commit hash одоогоор байхгүй (commit хийгээгүй).

Гишүүн	Хариуцах ажил (ойлгож хамгаалах хэсэг)	Commit/file
B200900028 Ө. Чинзориг	Shim: `list.h`, `kmalloc/kfree`, санах ойн бүртгэл	`src/shim/`
B221910021 Э.Төгсбаяр	Кернел модулиуд (Part I, Part II), Kbuild	`src/kernel/`
B231910034 Т. Буянтөгс	Engine: команд, JSON, `kds_verify`	`src/engine/`
B232270805 Г. Үлэмж	UI + python сервер	`src/ui/`
B241940102 Б. Лхамрагчаа	Тест (unit, cases, server)	`tests/`
B242270075 Б. Идэрбат	Баримт: README, Report, Manual, GitHub/Teams пост	`README.md`, `docs/`

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	`color example` (номын жишээ)	4 зангилаа, violet (138, 43, 226), kmalloc 4 / kfree 0; `color clear` дараа амьд 0
2	`collatz run start=27`	112 зангилаа, 111 алхам, оргил 9232; `start=837799` → 525 зангилаа, оргил 2,974,984,576
3	`color add_tail red=256 green=0 blue=0`; `collatz run start=0`; `stack pop` (хоосон)	E_RANGE; E_RANGE; E_EMPTY — төлөв, тоолуур өөрчлөгдөхгүй
4	Санах ойн алдааг kmalloc байрлал бүрд үүсгэх (collatz 1,250 + color 4)	`-ENOMEM`, өмнө авсан бүгд чөлөөлөгдөнө, leak 0
5	Эвдэрсэн 10 жагсаалт (`next`/`prev`/тоо/poison/тойрог/self-loop)	Шалгагч алдааг барина, гацахгүй; төлөвт `"?"`
6	Жинхэнэ кернелд (багийн гишүүн): `insmod` color_list; `insmod` collatz `start=27`; `dmesg`; `rmmod`	Өнгө 4 мөр (violet эхэнд); Collatz 112 утга, оргил 9232 (индекс 77); `insmod`/`rmmod` алдаагүй

Хавсралт Б. Файлууд ба хэмжээ

Хүснэгт Б1. Файлууд ба хэмжээ (wc -l)

Зам	Үүрэг	Мөр
`src/shim/linux/*.h` (7 файл)	header shim; өөрийн `list.h`	86
`src/shim/kshim.{h,c}`	`kmalloc`/`kfree`, тоолуур	106
`src/kernel/{color_list.c,collatz.c,Makefile}`	Жинхэнэ модулиуд + Kbuild	107
`src/engine/{kds_core.{h,c},main.c}`	Команд задлагч, шалгагч, JSON	406
`src/ui/server.py`	HTTP сервер, engine-ийн менежер	263
`src/ui/static/*`	UI (HTML, CSS, JS)	514
`Makefile`	build-ийн зорилтууд	75
`tests/*.c|py|h`	Unit, differential, case, server	1,315
`tests/ui/*.py`	Браузерийн шалгалт (CDP)	270